LIBFT

Defense guide

Makefile · compilation · memory · undefined behavior

To understand, not to memorize: every answer comes from an underlying rule.

All the experiments and terminal outputs in this guide were actually run.

What you will master

- libft Makefile: what each line does, why that order, and what happens if you change anything (including libft.a → libft.c).
- Compilation and linking: from .c to libft.a, flags, and how to identify each error.
- Headers and guards, memory (segfault, double free, leaks), and undefined behavior.
- The difficult functions (memmove, strlcat, split, itoa, lists) with their reasoning and a bank of defense questions.

Table of contents

How to use this guide 4

How it is structured . . . 4

Box legend . . . 4

How to study it (realistic plan) . . . 4

1 · From .c to libft.a: the complete map 6

1.1 What libft.a really is . . . 6

1.2 The four stages of cc . . . 6

1.3 Symbols: what the linker sees . . . 6

1.4 Why one function per file (and why .a) . . . 7

1.5 How libft.a is used from another program . . . 7

2 · Makefile in depth 9

2.1 The idea: a dependency graph and a comparison of dates . . . 9

2.2 Your Makefile, line by line . . . 9

2.3 Why that order? What matters and what doesn't . . . 11

2.4 Variables and symbols you need to know how to read . . . 11

2.5 Relinking: what it is and how to avoid it (bonus included) . . . 12

2.6 Why .o depends on libft.h . . . 13

2.7 .PHONY: targets that aren't files . . . 13

2.8 Lab: "what if I change...?" (everything tested) . . . 13

2.9 The star case: "what if I change libft.a to libft.c?" . . . 14

2.10 They ask you to modify the Makefile live (with solution) . . . 15

2.11 Makefile question bank . . . 15

3 · Compilation, flags, and errors 17

3.1 The flags they're going to ask you about . . . 17

3.2 Error map: message → stage → cause . . . 17

4 · Headers, guards, and visibility 19

4.1 What goes in the .h (and what doesn't) . . . 19

4.2 How guards work . . . 19

4.3 Details they ask about . . . 19

5 · Memory: stack, heap, and all its errors 21

5.1 Where everything lives . . . 21

5.2 The malloc and free contract . . . 21

5.3 The errors, tested one by one . . . 22

5.4 Segfault: what it really is . . . 22

5.5 Double free and use-after-free . . . 23

5.6 Memory leak: what it is and what types there are . . . 23

5.7 The tools (and how to read them) . . . 23
5.8 Memory discipline applied to libft . . . 24

6 · Undefined behavior (UB) **25**
6.1 Three things that get confused . . . 25
6.2 Why UB is dangerous: the compiler “assumes” it does not occur . . . 25
6.3 Catalog of UB that you have to recognize . . . 25
6.4 Where UB is in libft (hot spots) . . . 26

7 · Functions, one by one **27**
7.0 Three concepts that appear everywhere . . . 27
7.1 Part 1 · libc functions . . . 27
7.1b Part 2 · additional functions . . . 28
7.1c Bonus · linked lists . . . 28
7.2 In-depth analysis of the difficult ones . . . 29

8 · Defense question bank **35**
8.1 Library and compilation . . . 35
8.2 Memory . . . 35
8.3 C, UB, and function details . . . 36

9 · Subject standard and rules (review summary) **37**
9.1 Standard: what is most often violated . . . 37
9.2 libft subject: what is checked . . . 37

10 · Defense protocol **38**
10.1 Checklist before entering (with what you must see) . . . 38
10.2 During the defense . . . 38
10.3 Self-assessment: what if...? (answer out loud) . . . 38
10.4 Final 30-minute laboratory . . . 39

Appendix A · Command cheat sheet **40**

Appendix B · One-line glossary **41**

How to use this guide

A libft defense is not passed by reciting: it is passed by reasoning. The evaluator almost it always makes the same move: it lets you explain something and, once you have recited it from memory, it asks «what if I change this?» . This guide is designed so that question no longer scares you, because you will understand the underlying rule from which each answer follows.

How it is structured

- Each topic follows the same order: idea → image → experiment → defense phrase .
- The terminal outputs you see (gcc, make, glibc, and AddressSanitizer errors) are real :

  I ran them on Linux with gcc 13 and GNU make for this guide. On macOS/clang, the wording of the message changes slightly; the cause is the same.
- The example implementations (memmove, strlcpy, strlcat, atoi, calloc, split, itoa, lstmap...)

  I compiled them with -Wall -Wextra -Werror and compared them against libc with AddressSanitizer and UBSan: they produced no errors or leaks.
- When a rule depends on the version of the subject or the Standard (e.g. if wildcard is prohibited),

  I'll flag it for you: confirm it in your current subject. The rules change between versions, and I don't want you to memorize something that no longer applies.

Box legend

The key idea

The underlying rule that explains everything else in the topic. If you remember only one thing, let it be this.

They ask you this

A real defense question (or a very similar one).

Trap

What usually gets your project sent back.

Try it yourself (2 minutes)

A short experiment in your terminal. Doing it is worth more than reading it ten times.

Say it this way during the defense

A ready-to-use 15–20-second answer, in your own words. Adapt it; don't recite it.

How to study it (realistic plan)

1. Session 1 – Makefile (chapter 2). Do the "what if I change...?" lab with your own Makefile. It is which returns more projects.
2. Session 2 – Compilation and headers (chapters 1, 3, and 4). Deliberately cause each compilation error compilation and linking.
3. Session 3 – Memory (chapter 5). Cause a segfault, double free, leak, and overflow; read them with -fsanitize=address .
4. Session 4 – Undefined behavior and functions (chapters 6 and 7). For each function: what happens with NULL, 0, INT_MIN, overlap?
5. Session 5 – Mock exam (chapters 8 to 10). Answer the question bank aloud without looking, and do the "live" tests.

Golden rule

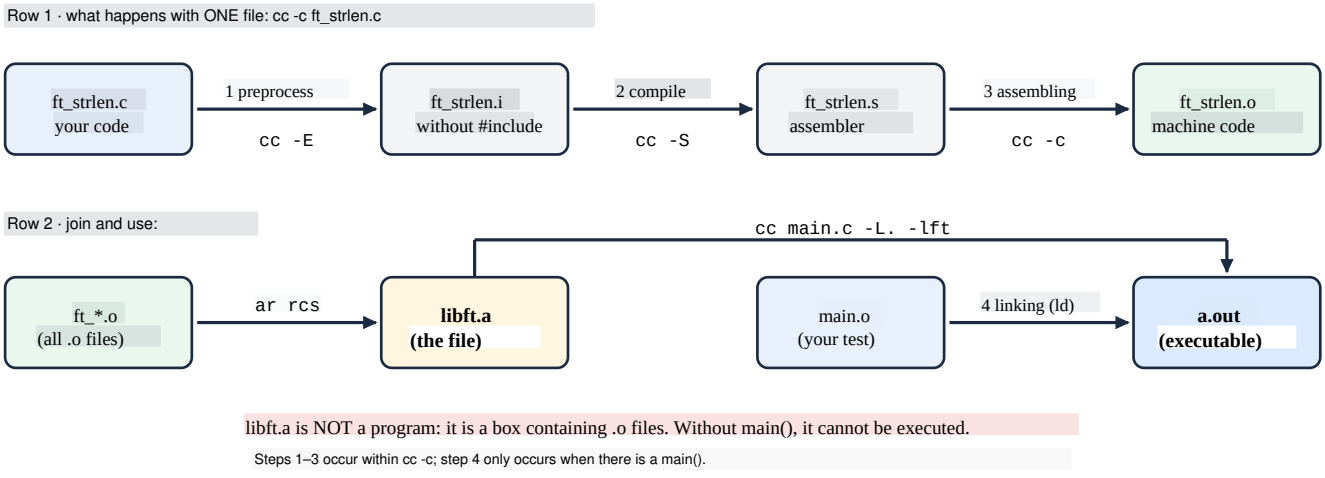
If you don't know something during the defense, don't make it up: reason out loud from the basic rule. Example: «I don't remember the exact flag, but I know that make compares dates, so if I change X, Y won't be regenerated because...». That scores much higher than silence or an invented fact.

1 · From .c to libft.a: the complete map

Before the Makefile, you need the map of what is being built. The Makefile is just a way to automate these steps.

1.1 What libft.a really is

A static library is a file (archive) that groups several .o object files and adds an index of symbols to them. It is like a tar designed for the linker. It is not a program, it has no main, and it is not executed; it is a box of already compiled functions waiting for someone to use them.



Complete pipeline: your source code becomes an object file, the object files are grouped into libft.a and, only when linking with a main, an executable appears.

1.2 The four stages of cc

Stage	Flag	What it does	What fails here
1 · Preprocess	`cc -E`	Processes lines with #: pastes the #include directives, replaces #define directives, evaluates #ifndef, removes comments. Output: pure C.	`Header not found ( fatal error: x.h: No such file ).`
2 · Compile	`cc -S`	Translates C into assembly. This is where types, `prototypes, and syntax are checked, and -Wall` -Wextra are produced.	`Syntax errors, implicit` declaration, warnings turned into errors by -Werror .
3 · Assemble	`cc -c`	Converts assembly into machine code: an .o file. Functions from other files remain as unresolved symbols.	Rare in libft.
4 · Linking	ld (via cc)	Join the .o files and libraries, resolve each symbol pending with its definition and produce the executable. It requires a main .	undefined reference to ... , multiple definition of ... .

The key idea

Compiling works file by file and only needs to know what each function is called and what signature it has (the prototype from the .h). Linking requires the code to actually exist. Therefore: function without a prototype = compilation error; prototype without a function = linking error.

1.3 Symbols: what the linker sees

A symbol is the name of a function or global variable inside a .o. nm lists them. These two letters are the ones that matter:

```
$ nm libft.a

ft_strlen.o:
0000000000000000 T ft_strlen <- T = defined here (in the code section)

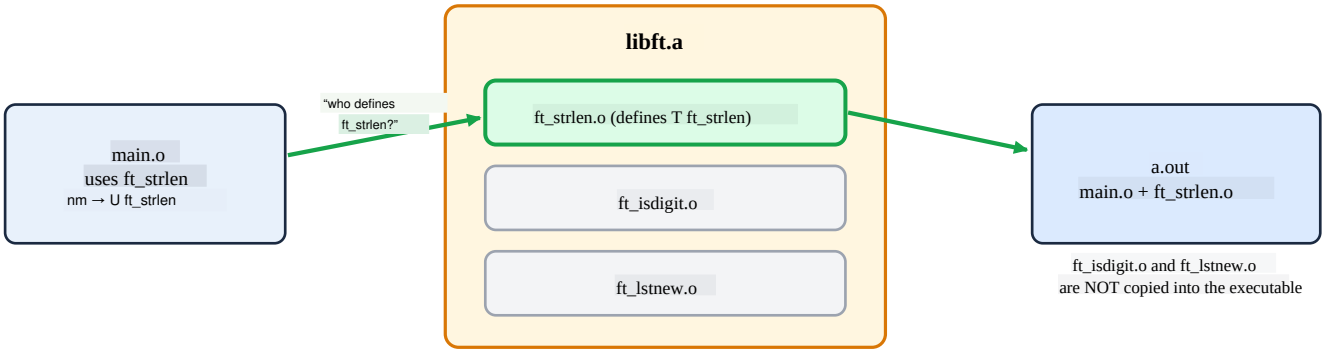
ft_isdigit.o:
0000000000000000 T ft_isdigit

$ nm main.o | grep ft_
U ft_strlen <- U = Undefined: it uses it but does not define it
```

The linker searches for each U for some object that has the same function in T. If it does not find it: undefined reference.

1.4 Why one function per file (and why .a)

When linking with a static library, the linker only extracts the members it needs. I checked it: a library with ft_strlen, ft_isdigit, and ft_lstnew linked with a main that only uses ft_strlen leaves only ft_strlen in the executable (nm a.out | grep ft_ only shows that one).



The linker asks “who defines ft_strlen?” and copies only that member from the archive.

	Static (.a)	Dynamic (.so / .dylib)
When it joins	When linking: the code is copied into the executable.	At runtime: the system loads it when launching the program.
Executable	Larger, standalone (it does not depend on the library on disk).	Smaller, but it needs to find the .so when executing.
Update the library	The programs that use it must be recompiled.	It is enough to replace the .so (if the interface does not change).
It is created with	`ar rcs libft.a *.o`	cc -shared -fPIC ... (this is not what libft requests)

1.5 How to use libft.a from another program

```
cc main.c -I. -L. -lft -o prueba # usual method
cc main.c libft.a -I. -o prueba # equivalent: you pass the path to the .a directly
```

Flag	It means	If it is missing...
`-I.`	Look for the headers in the current directory as well current.	With #include "libft.h" it usually works the same if it is alongside the .c file; it fails if the header is in another folder.
`-L.`	Look for libraries in the current directory.	`cannot find -lft`
`-lft`	**Links with libft.a: the name is lib + ft + .a.**	`undefined reference to ft_...`

Trap

`Order matters with static libraries. I tried cc -L. -lft main.c and it fails with undefined reference to` `ft_strlen': the linker processes from left to right; when it reaches libft.a, it has not yet seen what main.o needs, so it extracts nothing. Rule: first the .c / .o files that use them, then the libraries.

Try it yourself (2 minutes)

With your libft: make, then ar t libft.a (lists the members), nm libft.a | grep ' T ' (defined functions), and compile a minimal main with cc main.c -L. -lft -I. . Then try reversing the order and look at the error.

Say it this way in the defense

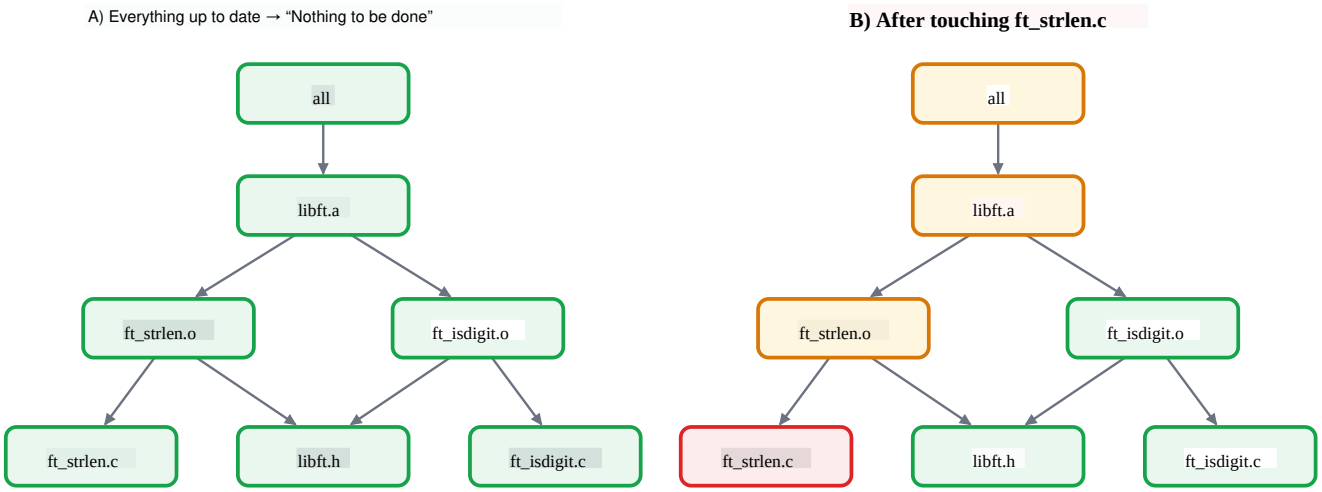
"libft.a is a static library: a file created with ar that groups my object files and a symbol index. Each function goes in its own .c file so that, when linking, only the ones the program uses are copied into the executable. No it has main, so it is not executed: it is linked with another program using -L. -lft ."»

2 · Makefile in depth

The Makefile is what is asked about most and where the most projects are sent back. The good news: the entire Makefile follows from a single rule. If you understand it clearly, you can deduce any answer instead of memorizing it.

2.1 The idea: a dependency graph and a comparison of dates

> **The key idea**
>
> make does not “execute a script.” It reads a graph: each target depends on prerequisites. To build a target, it does this:
>
> - 1. First, recursively builds each prerequisite.
> - 2. If the target does not exist, or any prerequisite has a more recent modification date, execute the recipe.
> - 3. Otherwise, it says «Nothing to be done» and does nothing.
>
> Everything else (no relink, recompiling only what has changed, .PHONY...) is a consequence of this.



Left: everything is newer than what depends on it, so make does nothing. Right: after touch ft_strlen.c is newer than its .o → that .o is rebuilt → the .o is newer than libft.a → libft.a is rebuilt. ft_isdigit.o is not touched.

```
$ make # 1st time: builds everything
$ make # 2nd time:
make: Nothing to be done for 'all'.
$ touch ft_strlen.c && make
cc -Wall -Wextra -Werror -c ft_strlen.c -o ft_strlen.o
ar rcs libft.a ft_strlen.o ft_isdigit.o # only that .o was recompiled
```

2.2 Your Makefile, line by line

This is a complete and correct Makefile for libft (the one I use as a reference throughout the guide; ⇥ marks a real TAB). I tested it: it does not relink, recompiles only what is necessary, and respects header changes.

```
NAME = libft.a
CC = cc
CFLAGS = -Wall -Wextra -Werror
SRCS = ft_isalpha.c ft_strlen.c ft_memmove.c ft_split.c
BSRCS = ft_lstnew.c ft_lstadd_front.c ft_lstmap.c
OBJS = $(SRCS:.c=.o)
BOBJS = $(BSRCS:.c=.o)

all: $(NAME)
```

```
$(NAME): $(OBJS)
	ar rcs $(NAME) $(OBJS)

%.o: %.c libft.h
	$(CC) $(CFLAGS) -c $< -o $@

bonus: .bonus

.bonus: $(OBJS) $(BOBJS)
	ar rcs $(NAME) $(OBJS) $(BOBJS)
	touch .bonus

clean:
	rm -f $(OBJS) $(BOBJS) .bonus

fclean: clean
	rm -f $(NAME)

re: fclean all

.PHONY: all bonus clean fclean re
```

Line	What it does	Why it is there / what happens if it is missing
`NAME = libft.a`	Stores the library name in a variable.	Avoid repeating it. It is just a chain: make does not know that it “is” a library (see 2.9).
`CC = cc`	Compiler to use.	`Allows changing it from outside: make CC=clang` . cc = default compiler of the system.
`CFLAGS = -Wall -Wextra -Werror`	Compilation flags.	`The subject requires them. With -Werror` any warning breaks the compilation.
`SRCS / BSRCS`	Explicit list of source files (mandatory and bonus part).	Explicit on purpose: nothing gets included by accident (see wildcard in 2.8).
`OBJS = $(SRCS:.c=.o)`	Substitution reference: change the suffix .c to .o in each word of SRCS.	That way you don't maintain two lists. If you add a .c to SRCS, its .o appears automatically.
`all: $(NAME)`	Default target: depends on the library.	It's the first rule, so it's what make runs by itself.
`$(NAME): $(OBJS)`	«libft.a depends on all .o files».	If any .o is newer than libft.a (or libft.a doesn't exist), the recipe is executed.
`ar rcs $(NAME) $(OBJS)`	Creates/updates the file. r inserts or replaces members; c creates without warning if it does not exist; s writes the symbol index (like ranlib).	Without s on macOS, the linker may complain that the file doesn't have index.
`%.o: %.c libft.h`	Pattern rule: «any X.o depends on X.c and libft.h». % is the common part (the stem).	A single rule builds all .o files. Adding libft.h makes changing the header recompile everything (see 2.6).
`$(CC) $(CFLAGS) -c $< -o $@`	$< = first prerequisite (the .c); $@ = the target (the .o). -c = compile without linking.	With $^ gcc also receives the .h file and fails: `cannot specify '-o' with '-c' ...` with multiple files (tested). Without -c: undefined reference to main .
`bonus: .bonus`	**The bonus target depends on a** marker file.	Trick to avoid relinking (see 2.5).
`clean`	Deletes the .o files (and the marker).	rm -f: -f prevents an error if they do not exist.
`fclean: clean`	First performs clean and also deletes the library.	Reuse the clean rule instead of repeating it.
`re: fclean all`	Start from scratch: fclean, then all.	The left→right order is essential (see 2.3 and 2.8).

Line	What it does	Why it is there / what happens if it is missing
`.PHONY: ...`	Declares goals that are not . files.	`Without this, a file named clean` would block the rule (see 2.7).

2.3 Why that order? What matters and what doesn't

This is the trick question: "why are the rules in that order?". The honest answer is that almost all of the order is convention; only a few places matter. Knowing how to distinguish them shows that you understand make.

What order?	Does it matter?	Why
`Position of all`	**YES**	The default goal is the first rule in the file. I tried to put clean before all: a bare make executed clean and built nothing. Fix: .DEFAULT_GOAL := all.
Order of the remaining rules	No	make builds the complete graph before executing anything; it does not execute in the order in which the rules appear.
Order of prerequisites (re: fclean all)	YES (in normal execution)	`make processes them from left to right. With re: all fclean` you build and then delete everything. Tested: you end up without libft.a.
With make -j (parallel)	**It breaks**	With -j8 re: fclean all launches fclean and all at the same time; in my test `it finished without libft.a. Robust solution: re: with recipe $(MAKE)` fclean and $(MAKE) all.
Order of variables	It depends	`= is lazy: it expands when used, so OBJS =` $(SRCS:.c=.o) works even if SRCS is defined afterward. := is immediate: with SRCS defined afterward, OBJS remains empty (I tested it: []).
`Order of the .o files within ar`	No	The members are independent. It would only matter with duplicate functions.
Order of -lft when linking	**YES**	After what uses it (see 1.5).
`.PHONY position`	No	It is a declaration; its location does not matter. It is placed at the end out of custom.
clean before fclean	No	Reading convention. fclean: clean works the same way in any position.

> **Say it this way in the defense**
>
> "Almost the entire order is convention because make builds a graph. It only matters that all is the first rule, because it is the default target, and the order of the prerequisites of re, because they are processed from left to right: fclean has to come before all."

2.4 Variables and symbols you need to know how to read

Symbol	Meaning	Example
`=`	Recursive/lazy assignment (evaluated when used).	`OBJS = $(SRCS:.c=.o)`
`:=`	Immediate assignment (evaluated at that point).	*X := $(Y) uses the value of Y now*
`?=`	Assign only if the variable was not defined.	`CC ?= cc (allows CC=clang` make)
`+=`	Add at the end.	`CFLAGS += -g`
`$@`	Goal name.	`In %.o: %.c: the ft_split.o`
`$<`	First prerequisite.	`The ft_split.c`
`$^`	All prerequisites (without repeating).	In $(NAME): $(OBJS): the list of .o

Symbol	Meaning	Example
`$?`	Prerequisites newer than the target.	Useful for incremental ar
`%`	Pattern rule wildcard (the stem).	`%.o: %.c`
`$(VAR:.c=.o)`	Suffix substitution in a list.	a.c b.c → a.o b.o
`@cmd`	Does not show the command before executing it.	`@echo hello`
`-cmd`	Ignores the error from that command and continues.	`-rm file`
\ (end of line)	Continue the line on the next one.	Long SRCS lists
`$(MAKE)`	Invoke make recursively (inherits flags such as -n, -j).	`$(MAKE) all`

> **Try it yourself (2 minutes)**
>
> Tools to see what make is thinking: make -n (shows the commands without executing them), make -B (forces rebuild everything), make --debug=b (explains why it rebuilds each target).

2.5 Relinking: what it is and how to avoid it (including the bonus)

Relinking = make runs commands (compile or archive) again when nothing has changed. During the defense, they will run make twice in a row: the second time it should say «Nothing to be done». When there is relinking, it is almost always due to one of these three causes:

Cause	Bad example	Why it relinks
False target rule with recipe	`all: $(OBJS)   ar rcs libft.a $(OBJS)`	«all» never exists as a file → make always thinks it needs to build it and runs ar each time.
The recipe creates a file with another name	%.o: %.c but -o writes to another path	make looks for the expected .o, does not find it, and always rebuilds it.
Prerequisite that changes only	Depending on a file that itself recipe touches/regenerates	It is always «newer» than the target.

The classic bonus problem

`With the usual rule bonus: $(OBJS) $(BOBJS) + ar rcs ..., since bonus is a phony target, make runs ar every time. I tested it: the second time I run make bonus, ar rcs is printed again` libft.a ... . That counts as relinking. Two solutions, both verified:

Solution A · marker file (the one in the reference Makefile): the actual target is a .bonus file that the recipe creates with touch .

```
bonus: .bonus

.bonus: $(OBJS) $(BOBJS)
	ar rcs $(NAME) $(OBJS) $(BOBJS)
	touch .bonus # now "bonus" has a real file with a timestamp

# 2nd time: make: Nothing to be done for 'bonus'.
```

Solution B · WITH_BONUS variable: the same target $(NAME) changes its prerequisites according to a variable.

```
ifdef WITH_BONUS
	ALL_OBJS = $(OBJS) $(BOBJS)
else
	ALL_OBJS = $(OBJS)
endif

$(NAME): $(ALL_OBJS)
	ar rcs $(NAME) $(ALL_OBJS)
```

```
bonus:
	@$(MAKE) WITH_BONUS=1 all
```

Verified: make bonus the 1st time compiles the bonus files and creates the .a; the 2nd time it says Nothing to 'all'; and a subsequent make does not remove them. Note: if you do not have a file called Makefile (e.g. you call it mk and use -f mk), the recursive $(MAKE) does not know which one to read.

2.6 Why the .o depends on libft.h

A change in the header (a prototype, a struct) affects the .c files that include it, but make does not read the C code: it only knows the dependencies that you write. I tested it: without libft.h as a prerequisite of %.o, I ran touch libft.h and make replied Nothing to be done. The old .o files remained compiled with the old version of the header: a silent bug that only appears at runtime.

```
%.o: %.c libft.h # simple and sufficient in libft (a single header)

# Professional alternative: let the compiler itself generate the dependencies
CFLAGS += -MMD -MP # creates ft_strlen.d with: ft_strlen.o: ft_strlen.c libft.h
DEPS = $(SRCS:.c=.d)
-include $(DEPS) # the '-' ignores the error if they do not yet exist
```

2.7 .PHONY: targets that are not files

make assumes that every target is a file. clean, all, or re are actions, not files. If by chance a file called clean exists and the rule has no prerequisites, make considers it “up to date.” I checked it: without .PHONY and with a clean file in the folder, make clean replied make: 'clean' is up to date. and did not delete anything. .PHONY tells make: “do not look at the disk, always execute the recipe.”

> **Trap**
> Do not put $(NAME) or the .o files in .PHONY: since they are «always out of date», you would force the relink you want to avoid.

2.8 Laboratory “what if I change...?” (everything tested)

This table is the best investment in the guide. Each row is a change an evaluator may ask you to make, with what actually happened when testing it and the underlying rule. Your job is to reproduce the first 10 on your machine.

If you change...	What happens (verified)	The underlying rule
`NAME = libft.c`	It works when building, but the result cannot be used, and fclean may delete your source. **Details in 2.9.**	ar ignores extensions; gcc and ld no.
all is no longer the first rule	make by itself executed clean and built nothing.	Default target = first rule (fix: `.DEFAULT_GOAL := all`).
`You remove .PHONY`	`With a clean file present: 'clean' is up` to date .	Without PHONY, make treats the name as a file.
`re: all fclean`	Build and then delete: you end up without libft.a.	Prerequisites from left to right.
`re: fclean all with make -j8 re`	In my test, it ended up without libft.a (they were run in parallel).	-j breaks the implicit order. Use $(MAKE) in the recipe.
`Remove -c`	undefined reference to `main' : cc tries to create an executable from each .c file.	-c = stop after assembling.
Remove libft.h from the rule `%.o`	You touch the header and make recompiles nothing.	make only knows what you write.
Delete the entire %.o rule	It keeps working: make has an implicit rule that uses $(CC) $(CFLAGS) -c . You lose the header dependency.	Built-in implicit rules.

If you change...	What happens (verified)	The underlying rule
`SRCS = $(wildcard *.c)`	main.c ended up inside libft.a.	wildcard takes everything that exists in the folder.
You add -g to CFLAGS	make does not recompile (the .o files are up to date). It is necessary to run make re.	make only compares dates, not flags.
`touch ft_strlen.c`	It recompiles only that .o and rebuilds libft.a.	Graph + dates.
`touch libft.h`	It recompiles all the .o files.	All depend on the header.
You manually delete an .o	Only that one is recompiled and libft.a is rebuilt.	The .o is missing → it needs to be built.
You delete libft.a and leave the .o files	Only ar rcs is executed (the .o files are up to date).	Only the objective is missing.
ar rc (without s)	On my Linux system it linked successfully because GNU ar also creates the index; on macOS, s or ranlib is needed.	s creates the index.
OBJS := before defining SRCS	OBJS remains empty; libft.a is created with no members.	Immediate vs lazy.
Spaces instead of TAB	`*** missing separator. Stop.`	Recipes start with TAB.
rm without -f in clean	`The 2nd time: rm: cannot remove ...: No such` file and make aborts (Error 1).	make stops at the first command that fails.
`$(CC) -o $(NAME) $(OBJS)` instead of ar	It tries to create an executable: it fails because there is no main; and even if it worked, it would not be a library.	libft.a is an archive.
`CC = clang / gcc`	Same result in this project.	cc = system default compiler.

2.9 The star case: “what if I change libft.a to libft.c?”

It is a classic question because it seems absurd and reveals whether you understand the difference between the name and the content. I ran it exactly as is:

```
$ make # with NAME = libft.c
cc -Wall -Wextra -Werror -c ft_strlen.c -o ft_strlen.o
cc -Wall -Wextra -Werror -c ft_isdigit.c -o ft_isdigit.o
ar rcs libft.c ft_strlen.o ft_isdigit.o # ar doesn't complain: it creates a FILE called libft.c

$ file libft.c
libft.c: current ar archive # the content is an archive, the name lies

$ cc main.c libft.c -I. -o prueba # gcc decides "this is C" based on the .c extension
libft.c:1:1: error: expected identifier or '(' before '!' token
1 | !<arch> # the 1st line of every archive is literally !<arch>

$ cc main.c -L. -lft -I. -o prueba
/usr/bin/ld: cannot find -lft # -lft looks for "libft.a", not "libft.c"

$ make fclean # deletes $(NAME) = libft.c
```

And the worst case: if a libft.c with actual source code already exists (e.g. you had it under that name), ar refuses to overwrite it (file format not recognized), but make fclean executes rm -f $(NAME) and deletes it. I checked it: after fclean, the source file had disappeared.

> **The key idea**
>
> ar does not look at the extension; everything else does. • ar creates an archive with the name you give it. • gcc decides what do with a file based on its extension: .c → “it is C code, compile it.” • -lft looks for a file named lib **+ ft + .a (or .so). So the .a extension is a convention, but a convention on which** the compiler and linker depend.

> **Say it this way during the defense**
>
> «Make doesn't complain because NAME is just a string, and ar doesn't look at extensions: it creates an archive called libft.c.
> But then gcc, seeing the .c extension, tries to compile it as C and fails with an error on the first line, which is
> !<arch> ; and -lft only looks for libft.a. Also, fclean would run rm -f libft.c, risking deleting a real source file.»

2.10 They ask you to modify the Makefile live (with solution)

a) “Add a new function”

1. Create ft_xxx.c with #include "libft.h".
2. Add it to SRCS (if it does not fit on the line, use \ and continue below).
3. Add its prototype to libft.h.
4. Run make and check with nm libft.a | grep ft_xxx.

There is no need to touch the rules: the pattern rule %.o: %.c already works, and OBJS is derived from SRCS. And since you modified libft.h, make recompiles all the .o files (because of the header dependency).

b) “Add a test rule”

```
test: $(NAME)
	$(CC) $(CFLAGS) main.c -L. -lft -I. -o test_libft
	./test_libft

.PHONY: all bonus clean fclean re test
```

c) “Add a debug mode with sanitizers”

```
debug: CFLAGS += -g3 -fsanitize=address
debug: re
```

It is a target-specific variable: CFLAGS only changes when debug is built, and its prerequisites inherit it (re → fclean all → the .o files). Verified: the .o files are compiled with -g3 -fsanitize=address.

Remember that when linking your main, it also needs -fsanitize=address.

d) “Make the .o files go into an obj/ directory”

```
OBJDIR = obj
OBJS = $(addprefix $(OBJDIR)/, $(SRCS:.c=.o))

$(OBJDIR)/%.o: %.c libft.h | $(OBJDIR)
	$(CC) $(CFLAGS) -c $< -o $@

$(OBJDIR):
	mkdir -p $(OBJDIR)

clean:
	rm -rf $(OBJDIR)
```

The | $(OBJDIR) is an order-only prerequisite: it requires the folder to exist before compiling, but the folder changing its timestamp (each time you put a file inside it) does not trigger recompilations. Verified: second execution → Nothing to be done.

e) “Check that the rule works”

```
norm:
	norminette $(SRCS) libft.h
```

2.11 Makefile question bank

Question	Short answer
What does make do?	It reads a graph of targets and prerequisites and, by comparing modification dates, executes only the recipes for what is outdated or does not exist.

Question	Short answer
What happens if I run make twice ?	The second one does nothing (Nothing to be done) because libft.a is newer than all the .o files, and these are newer than their .c files and libft.h.
Why is all the first rule?	Because the default target is the first rule in the file; that is where it is decided what make does without arguments.
What is .PHONY?	List of targets that are not files. Prevents a file with that name block the rule.
What does ar rcs mean?	r inserts/replaces members, c creates the file without warning, s writes the index of symbols (equivalent to ranlib).
What is the difference between = and :=?	= is evaluated when using the variable (lazy); := is evaluated when defining it (immediate).
What are $@, $<, and $^?	Target, first prerequisite, and all prerequisites.
Why %.o: %.c libft.h?	A pattern rule builds any .o from its .c, and adding the header makes changing libft.h recompile (make does not read the C).
What is relinking and how do you avoid it with bonus?	Redo things without changes. With bonus, using a marker file (.bonus) or the WITH_BONUS variable.
Why not use wildcard?	It takes everything in the folder (tested: it put main.c into libft.a), it is not **reproducible and does not warn about missing files. Check your subject to see if it** is prohibited; in practice, evaluators reject it.
Why fclean: clean?	To reuse clean and add deleting the library without duplicating commands.
Difference between clean, fclean, and re?	clean deletes .o; fclean also deletes libft.a; re = fclean and then all (rebuild from scratch).
And re with make -j?	`It may fail because fclean and all run at the same time; this is fixed by cal` sequentially within the recipe.
Why -Wall -Wextra -Werror?	Wall and Wextra enable warnings; Werror turns them into errors so that questionable code does not pass. The subject requires them.
I delete libft.a but leave the .o files. What does make do?	Only run ar rcs: the .o files are up to date.
What if there is a file called all?	With .PHONY it ignores this and executes the rule; without .PHONY it might consider it up to date.
Why does the recipe start with TAB?	It is make syntax; with spaces it gives missing separator.
What do @ and - before a command?	@ does not display the command; - ignores its error and continues.
What is $(MAKE) used for?	Call make recursively, inheriting flags (-n, -j...) and the context.
I changed CFLAGS and it didn't recompile, why?	make only compares file timestamps, not the contents of variables. It needs make re.

3 · Compilation, flags, and errors

A good defensive trick: identify at which stage something fails. If you know whether an error is in preprocessing, compilation, linking, or execution, you already have half the answer.

3.1 The flags they will ask you about

Flag	What it does	Useful detail
`-Wall`	Enables a broad set of “common” warnings.	They are not «all»: the name is misleading.
`-Wextra`	Adds more warnings (e.g. comparisons signed/unsigned, unused parameters).	When size_t and int are mixed, it warns you.
`-Werror`	Turns every warning into an error: compilation stops.	Mandatory in the subject. A warning = project that does not compile = 0.
`-c`	Compiles and assembles, does not link: produces a .o.	`Without it: undefined reference to` main .
`-o name`	Output file name.	With -c and multiple input files it cannot be used.
`-I dir`	Extra directory in which to search for headers.	For #include "x.h" in another folder.
`-L dir`	Extra directory in which to search for libraries.	It is used with -l .
`-lft`	Links with libft.a (lib + ft + .a).	It comes after the files that use it.
`-g`	Adds debugging information (names, lines).	Essential for gdb/valgrind with useful line numbers.
`-O0 / -O2`	Optimization level (none / high).	With optimization, UB may behave differently.
`-fsanitize=address`	Instruments the program to detect memory errors at runtime.	Heap overflow, use-after-free, double free, leaks (Linux).
`-fsanitize=undefined`	Detects certain UB at runtime (signed overflow, shifts...).	UBSan.
`-E / -S`	Stop after preprocessing / after generating assembly.	They are useful for viewing each stage.
`-MMD -MP`	Generates .d files with the dependencies of headers.	See chapter 2.6.

3.2 Error map: message → stage → cause

Each row is an error I deliberately caused; the text is what gcc/ld printed.

Message	Stage	Typical cause	Fix
`fatal error: libft.h: No such file or directory`	Preprocessing	Header outside the path of search.	-The correct thing is to move the file.
`implicit declaration of function 'ft_nada' [-Werror=implicit-function-declaration]`	Compilation	You call a function without a prototype visible (the include or the declaration in the .h is missing).	Declare it in the header and include it.
`redefinition of 'struct s_list'`	Compilation	Header without guards included twice in the same unit.	Add guards (cap. 4).
`'x' is used uninitialized [-Wuninitialized]`	Compilation (warning)	Reading a variable without assigning it a value.	Initialize it.

Message	Stage	Typical cause	Array
`pointer of type 'void *' used in arithmetic [-Wpointer-arith]`	Compilation (with -pedantic)	Arithmetic with void * (extension of GNU, non-standard).	`Cast to unsigned` char * .
`undefined reference to `ft_nada'`	**Linking**	Prototype yes, definition no; or failure to link the library; or incorrect order of -l .	Define the function / add -lft at the end.
`multiple definition of `twice'`	**Linking**	Function defined (with a body) in a header included by two .c files; or the same function in two files.	Only prototypes in the .h; or static inline .
`undefined reference to `main'`	**Linking**	Compile without -c a file that does not have main.	Add -c .
`cannot find -lft`	**Linked**	libft.a is not in the -L paths.	-L. or build the library.
Segmentation fault (exit 139)	**Execution**	Invalid memory access.	Chapter 5.
`free(): double free detected` in tcache 2 (exit 134)	**Execution**	Freeing the same pointer twice.	Chapter 5.

The key idea

Compile = check file by file that what you write is consistent with the prototypes. Link = check that everything used exists somewhere. Execute = discover what neither of the two stages could see (memory, logic, UB).

Try it yourself (2 minutes)

Trigger these three with 3-line files: (1) call ft_nada(3) without declaring it; (2) declare it but do not define it; (3) define a function in a .h and include it from two .c files. If you can distinguish the three messages, you master half of the compilation questions.

Say it this way in the defense

«A compilation error means that my code is not consistent with the prototypes; a linking error means that one declared function is not defined or the library was not linked; and runtime errors, such as a segfault, are detected by only the system or a sanitizer. That's why I compile with -Wall -Wextra -Werror and test with -fsanitize=address .»

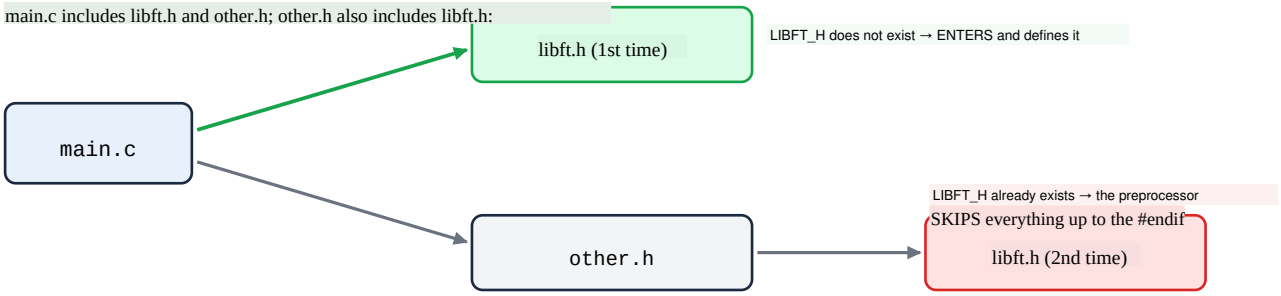
4 · Headers, guards and visibility

4.1 What goes in the .h (and what doesn't)

Yes, it goes in the .h	It does not go in the .h
`Prototypes: size_t ft_strlen(const char *s);`	Function bodies (causes multiple definition when linking).
`typedef and struct: t_list`	Global variable definitions.
The #include directives required by the prototypes themselves (e.g. <stddef.h> for size_t).	Internal helper functions (those are static inside their .c file).
Guards: #ifndef / #define / #endif.	Executable code of any kind.

4.2 How guards work

```
#ifndef LIBFT_H /* does the LIBFT_H macro already exist? NO -> keep reading */
# define LIBFT_H /* I define it: from now on, "I've already been here" */
# include <stddef.h>
size_t ft_strlen(const char *s);
#endif /* end: if the macro existed, the preprocessor skips everything up to here */
```



The first time, the macro does not exist and the content is included; any other inclusion in the same compilation finds it defined and skips the content.

What problem it solves: in C, you cannot define the same struct or typedef twice in the same compilation unit. If one header includes another, it is easy for the same header to arrive through two paths. I tested it with a struct s_list without guards, included directly and indirectly:

```
nog.h:1:16: error: redefinition of 'struct s_list'
1 | typedef struct s_list { int v; } t_list;
```

And I checked with cc -E that, with guards, including the same header three times leaves a single copy of the content (int once; appears once).

> **Trap**
> Guards do NOT prevent multiple definition. They provide protection within a compilation unit. If you define a function with a body in the header and a.c and b.c include it, each .c compiles it separately (the guards work in each one) and the linker sees the function twice: multiple definition of `twice' . I tried it.

4.3 Details they ask about

Question	Answer
`#include "x.h" vs #include <x.h>`	With quotation marks, it first searches next to the current file and then in the paths of includes; with <> it goes directly to the system paths (and those from -I). For your headers: quotation marks.

Question	Respuesta
#pragma once instead of guards	Hace lo mismo y es más corto, pero no es estándar (casi todos los compiladores lo soportan). El estilo del proyecto/Norma pide guards.
Macro name	Único y estable, por convención el nombre del fichero en mayúsculas: LIBFT_H . Si dos headers usaran la misma macro, el segundo quedaría ignorado .
Why # define with a space?	Norm style: nested directives are indented after the #.
struct s_list *next inside the struct itself	You must use the struct s_list tag: the t_list alias does not yet exist while the struct is being declared.
Why is a helper static?	Without static, the name is a global symbol from libft.a: it may conflict with user functions and remains exposed. With static, it only exists within its .c file (in nm it appears in lowercase: t).
What does extern do?	Declares that a variable/function exists in another file. For functions, it is optional (this is the default behavior).
And what about const in prototypes?	Promises that the function does not modify what is pointed to: const char *s. Documents intent and allows literals to be passed without a warning.

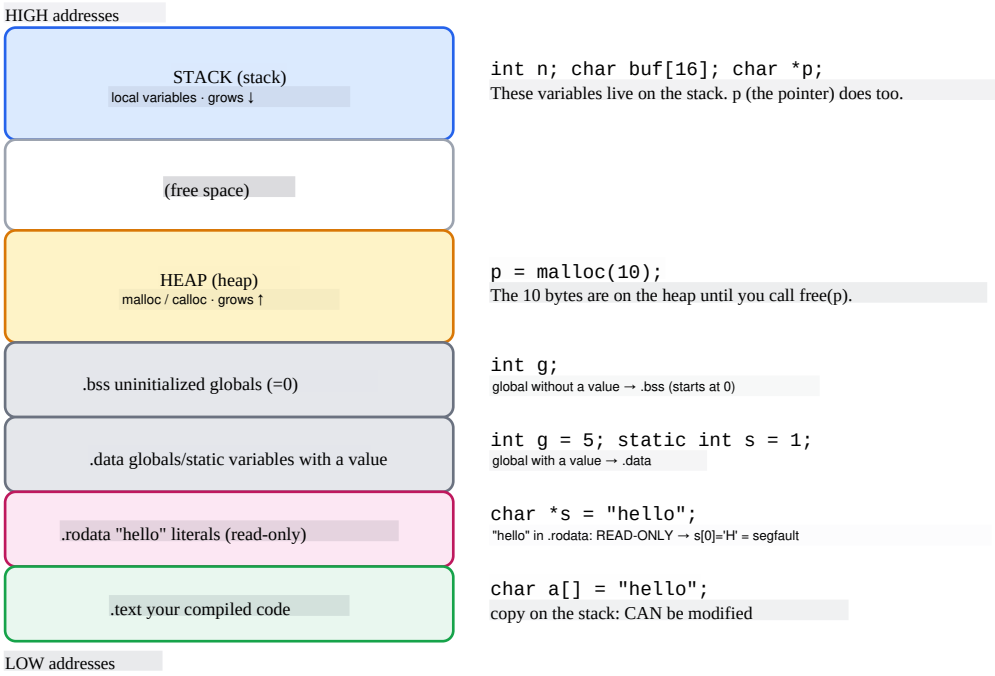
Dilo así en la defensa

«Los include guards evitan que el contenido de un header se procese dos veces en la misma unidad de compilación ; sin ellos, un struct o typedef incluido por dos caminos da redefinition . No sustituyen a poner solo declaraciones en el .h: una función con cuerpo en el header daría multiple definition al enlazar.»

5 · Memory: stack, heap, and all their errors

This is where the defense is won or lost. If you see where everything lives, errors (segfault, double free, leak...) stop being words and become drawings.

5.1 Where everything lives



Simplified map of a process's virtual memory.

	Stack (stack)	Heap
What it stores	Local variables, parameters, return addresses.	What you request with malloc / calloc.
Duration	Until the function ends (it is freed automatically).	Until you call free (or the program ends).
Size	Small and limited (typically a few MB).	Large, limited by the system.
Management	Automatic.	Manual: you are the owner.
Typical errors	Stack overflow (infinite recursion, arrays huge ones), return pointer to local.	Leak, double free, use-after-free, overflow, forgetting to check NULL.

Trap

A pointer and what it points to are two different things. In char *p = malloc(10); the variable p (8 bytes) lives on the stack; the 10 bytes live on the heap. When the function ends, p disappears but the 10 bytes remain allocated: if you did not call free, that is a leak.

5.2 The malloc and free contract

- **malloc(n) returns NULL if it fails: it must always be checked. It returns uninitialized memory** (garbage). calloc returns it zeroed.
- **free(p) is only valid with a pointer exactly equal to the one returned by malloc (not p + 1) and only once.** free(NULL) is valid and does nothing.
- malloc(0) is implementation-defined: it may return NULL or a unique freeable pointer. Your code must not depend on either one.

- Golden rule: every malloc has an owner and exactly one free . If you don't know who frees it, there's a bug.
- If a function allocates several things and one fails halfway through, it must free what has already been allocated before return NULL (key in ft_split and ft_lstmap).

5.3 The errors, tested one by one

Each row is a 2-line program that I ran. Notice the last column: several of the most serious errors make no noise.

Error	What it is	What happens when executing it (without tools)
Segfault (null deref)	strlen(NULL): reading address 0.	Segmentation fault · exit 139
Segfault (literal)	char *s = "hola"; s[0] = 72; → writing to .rodata (read-only).	Segmentation fault · exit 139
Stack overflow	Endless recursion with a local array large.	Segmentation fault · exit 139
Double free	`free(p); free(p);`	`free(): double free detected in tcache 2` · Aborted · exit 134
Invalid free	free(p + 1): pointer not returned by malloc.	free(): invalid pointer · Aborted · exit 134
Memory leak	p = malloc(42); and exit without free.	Nothing. exit 0. Invisible without a tool.
Use-after-free	Read p[0] after free(p) .	In my test it printed a garbage character. exit **0. No error.**
Heap overflow	malloc(4) and write 9 bytes.	Nothing. exit 0. Corrupts neighboring memory in silence.

> **The key idea**
> "It hasn't crashed" does not mean "it's fine." A leak, use-after-free, and heap overflow all resulted in exit 0. Only one detector (AddressSanitizer, valgrind) can see them. That is why the defense asks so much about memory: it is the part where the code "works" and is still wrong.

5.4 Segfault: what it really is

Every address your program uses is virtual. The CPU (its MMU) translates it into physical memory using page tables. If the address is not mapped for your process (such as 0), or the page exists but does not allow that operation (writing to a read-only page, executing data), the CPU raises an exception, and the kernel sends your process the SIGSEGV signal. The default action is to terminate the program (and optionally dump a core).

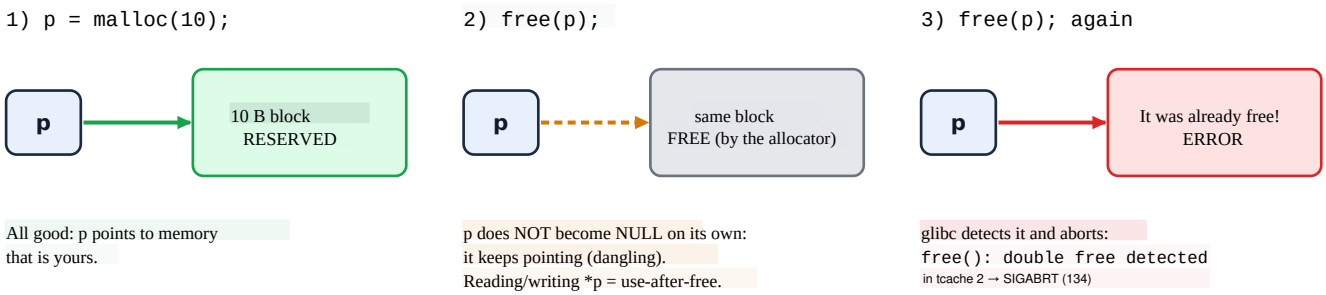
- Exit 139 = 128 + 11 (SIGSEGV is signal 11). Exit 134 = 128 + 6 (SIGABRT, what glibc throws when detect a double free).
- Typical causes: dereferencing NULL , an uninitialized or already freed pointer, an out-of-range index that it lands on an unmapped page, writing to a literal, infinite recursion (stack overflow).
- Important: a small overflow on the heap almost never causes a segfault (the page remains mapped); therefore that is detected poorly without tools.

```
$ cc -g main.c -o prueba # -g = debugging information
$ gdb ./prueba
(gdb) run
Program received signal SIGSEGV... <- tells you the line where the failure occurred
(gdb) bt <- backtrace: who called whom
```

Say it this way in the defense

"A segfault is a memory access violation: my program uses a virtual address that is not mapped or that it does not have permission to access (for example, 0 or a read-only area). The MMU detects it, the kernel sends SIGSEGV to the process, and it terminates with code 139. I debug it with gdb or AddressSanitizer."

5.5 Double free and use-after-free



If NOT detected: the block enters the free list twice, and two future malloc() calls they return the SAME address → two "owners" overwriting each other's data.

`Fix: free(p); p = NULL; (free(NULL) is valid and does nothing)`

After free(p), p is not set to NULL on its own: it continues pointing to memory that is no longer yours.

Trap

The underlying problem is the dangling pointer. A double free is simply calling free with a dangling pointer. The practical defense is free(p); p = NULL; → a second free(NULL) is harmless, and a subsequent access to p fails clearly rather than silently.

In libft, it appears above all in ft_lstclear (if you free(lst) and then read lst->next, it is a use-after-free: save next before freeing it) and in ft_split (freeing an already freed pointer when cleaning up after a failure).

5.6 Memory leak: what it is and what types there are

There is a leak when you allocate memory and lose all the pointers that point to it without having freed it: you can never free it again. The program works, but it consumes memory that does not come back.

Category (valgrind)	What it means	Gravedad
definitely lost	No pointer to that block remains.	Leak real.
indirectly lost	It is only pointed to by blocks that have already been lost (e.g., the nodes of a list whose first node was lost).	Leak real (consecuencia de otro).
possibly lost	There is only one pointer to the interior of the block, not to its start.	Sospechoso; casi siempre un bug.
still reachable	When the program ended, there was still a valid pointer (e.g., a global).	No es un leak estricto, pero en la defensa se espera 0.

AddressSanitizer (LeakSanitizer) reports it like this (real output):

```
==428==ERROR: LeakSanitizer: detected memory leaks

Direct leak of 42 byte(s) in 1 object(s) allocated from:
#0 0x7f2ac90fd9c7 in malloc ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:69
#1 0x55637e3ed19e in main /home/claude/lab2/leak.c:2 <- where it was allocated

SUMMARY: AddressSanitizer: 42 byte(s) leaked in 1 allocation(s).
```

5.7 The tools (and how to read them)

```
# AddressSanitizer (verified in this guide)
cc -g -O0 -fsanitize=address main.c libft.a -I. -o prueba && ./prueba

# valgrind (Linux; not available on this machine, but it is the standard for defense)
cc -g main.c libft.a -I. -o prueba
valgrind --leak-check=full --show-leak-kinds=all ./prueba

# macOS (valgrind usually does not work on recent versions)
leaks --atExit -- ./prueba
```

Two real AddressSanitizer outputs so you can recognize the pattern “what happened · where · who reserved/freed”:

```
ERROR: AddressSanitizer: attempting double-free on 0x502000000010 in thread T0:
#1 ... in main dfree.c:2 <- second free (the erroneous one)
0x502000000010 is located 0 bytes inside of 10-byte region
freed by thread T0 here:
#1 ... in main dfree.c:2 <- first free
```

```
ERROR: AddressSanitizer: heap-buffer-overflow on address 0x502000000014
WRITE of size 9 at 0x502000000014 thread T0
#0 ... in memcpy
#1 ... in main hof.c:3 <- who wrote too much
0x502000000014 is located 0 bytes after 4-byte region <- it went exactly past the end
allocated by thread T0 here: #1 ... in main hof.c:3
```

5.8 Memory discipline applied to libft

Rule	Where it is applied
Check every malloc/calloc.	ft_calloc, ft_strdup, ft_substr, ft_strjoin, ft_strtrim, ft_split, ft_itoa, ft_strmapi, ft_lstnew, ft_lstmap.
If it partially fails, free what was allocated and return NULL.	ft_split: frees each substring and the array. ft_lstmap: from the newly created content and ft_lstclear of the new list.
Save next before freeing a node.	ft_lstclear.
Always allocate len + 1 (the '\0').	ft_strdup, ft_substr, ft_strjoin, ft_itoa, ft_split.
Do not free what is not yours.	ft_lstdelone frees the node and calls del on the content; it does not touch anything else.
Do not return pointers to local variables.	Any function that returns char *: always heap memory.
Set to NULL after freeing.	ft_lstclear leaves *lst = NULL.

They ask you

- What is a memory leak and how do you detect it?
- What happens if I free the same pointer twice? And what if I set it to NULL beforehand?
- Where does the pointer live and where does what it points to live in char *p = malloc(10)?
- Why does s[0] = 'H' in a literal cause a segfault but not in a char array a[]?
- What does your ft_split do if the third malloc fails?

6 · Undefined behavior (UB)

“Undefined behavior” is the concept most often used to scare people in a defense. In reality, it is a simple idea with a serious consequence.

6.1 Three things that are confused

Term	What is	Example
Undefined behavior	The standard imposes no requirements: it may work, produce an unusual value, crash, or break with another compiler/flag.	*NULL, signed int overflow, out-of-bounds access, double free.
Unspecified	*The standard allows several options and does not require* documentation of which one is chosen.	Evaluation order of the arguments of a function.
Implementation-defined	**Several options, and the implementation must** document which one it chooses.	signed or unsigned char; malloc(0); int / long size.

6.2 Why UB is dangerous: the compiler “assumes” it does not occur

A compiler optimizes under the assumption that the program never invokes UB. If it does, its assumptions can turn against you. A real example I ran:

```
int always_bigger(int x) { return x + 1 > x; }

x = INT_MAX; always_bigger(x) -> 1 (with -O0 and -O2, gcc 13)
```

Mathematically, with INT_MAX, that should be false if the value “wraps around.” But signed integer overflow is UB, so the compiler may conclude that x + 1 > x is always true and not even execute the addition. Moral: with UB, there is no “what the machine would do,” only what whatever the compiler feels like. And that is why “it works for me” is not a valid defense: it may fail on the evaluator’s machine.

Another real fact: unsigned integer overflow (unsigned , size_t) is defined (it wraps back to 0, modulo 2 n).

6.3 Catalog of UB you have to recognize

UB	Example	How to avoid it / detect it
Dereference NULL or an invalid pointer	`*p with p == NULL`	Check pointers; ASan.
Out-of-bounds access	buf[10] in an array of 10; read beyond the end from '\0'.	Review indices and the +1 for the terminator; ASan.
Use-after-free / double free	Ch. 5.	p = NULL after free; ASan.
Signed integer overflow with	INT_MAX + 1 ; -INT_MIN ; abs(INT_MIN) .	Use an intermediate long or check beforehand; -fsanitize=undefined.
Read an uninitialized variable	int x; printf(x) (tested: warning from -Wall).	Always initialize; -Wall -Wextra.
Modify a string literal string	`char *s = "hello"; s[0] = 72;` (tested: segfault).	Use char s[] = ... if it must be modified.
`Overlap in memcpy`	memcpy(buf+2, buf, 6) .	Use memmove.
Pass NULL to libc functions with n = 0	memcpy(NULL, NULL, 0) is technically UB.	Consider a guard (see 6.4).
Return pointer to local variable local	`char buf[8]; return buf;`	Allocate on the heap.

UB	Example	How to avoid it / detect it
Subtracting/comparing pointers from different objects	p1 - p2 with different arrays.	Avoid it.
Out-of-range shift	1 << 32 in a 32-bit int.	Check widths; -fsanitize=undefined .
`Arithmetic with void *`	GNU extension, non-standard (tested with -pedantic).	Cast to unsigned char * .

6.4 Where the UB is in libft (hotspots)

Function	Risk	What to do / what to answer
ft_strlen, ft_strchr ...	With NULL, the original libc also fails.	It is UB in the original; imitating it (segfault) is defensible. The important thing is being able to explain it.
`ft_memcpy`	Overlap and/or NULL, NULL.	`If you protect it, make it exactly if (!dst` && !src) return (dst); and know how to justify it.
`ft_memmove`	If you always copy forwards, with overlap you overwrite data before reading it.	Choose the direction according to dst < src (see 7.2).
`ft_atoi`	Signed int / long overflow = UB (in the original libc as well).	Explain that overflow is undefined; accumulating in long delays it but does not eliminate it.
ft_itoa , `ft_putnbr_fd`	-n with n = INT_MIN overflows.	Convert to long before negating, or print digit by digit.
`ft_calloc`	nmemb * size can overflow (without sign: defined, but reserves less than requested → heap overflow later).	Check nmemb > SIZE_MAX / size and return NULL.
`ft_substr`	`start >= strlen(s) or start + len` large.	`Return a reserved empty string; limit len` to the actual remainder.
`ft_strlcat`	Read dst beyond size if there is no '\0'.	Calculate the length of dst with the size limit (see 7.2).
ft_strnstr , ft_strncmp , `ft_memcmp`	Exceeding len or '\0'; comparing with signed char.	Truncate at len and at '\0'; compare as unsigned char .
`ft_lstclear`	Use-after-free when reading ->next after the free.	Save next first.

Try it yourself (2 minutes)

Compile your libft with -g -fsanitize=address,undefined and pass it a main with the edge cases: INT_MIN , INT_MAX , empty string, len = 0 , start > strlen , overlap in memmove, ft_split with consecutive delimiters. If nothing is triggered, you're in very good shape.

Say it this way during the defense

«Undefined behavior means that the standard does not define what should happen: it may work, fail, or produce strange results, and even the compiler may optimize assuming that it does not occur. That's why I avoid, for example, signed integer overflow in itoa using long, or overlap in memcpy using memmove, and I check with sanitizers.»

7 · The functions, one by one

Here you have first the review table of all the functions (what they must do and what trap each one hides) and then the in-depth analysis of the difficult ones, with code of reference. First, three concepts that explain half the table.

7.0 Three concepts that appear everywhere

Concept	What you need to know
`size_t`	Unsigned type for sizes (on my machine 8 bytes: sizeof(size_t) = 8). Trick: while (i >= 0) with size_t i never ends (it is always ≥ 0). And while (n--) with size_t n is correct: for when n reaches 0.
unsigned char for compare	The sign of char depends on the implementation. Comparing with signed char makes the byte 0xFF negative. Tested: memcmp returned 254 (255−1, as unsigned char), while a[0] - b[0] with char gave −2. That is why memcmp, strncmp, memchr compare as unsigned char.
void * and bytes	You cannot perform arithmetic with void * or dereference it (GNU allows it, but it is not `standard: -pedantic warns). Inside ft_memcpy / ft_memmove, you convert to unsigned` char * and work byte by byte.

7.1 Part 1 · libc functions

Function	What it must do	Traps and details they ask about
`ft_isalpha / isdigit / isalnum`	They return nonzero if the character is letter / digit / alphanumeric; 0 if not.	int parameter. libc returns «not equal to zero», not necessarily 1.
`ft_isascii / isprint`	ascii: 0–127. print: 32–126 (printable).	Exact limits; 127 (DEL) is not printable.
`ft_strlen`	Number of characters before '\0'.	Returns size_t. It does not count the terminator.
`ft_memset`	Writes the byte (unsigned char)c to the first len bytes; returns b.	The second parameter is int but is used as a byte.
`ft_bzero`	Like memset to 0; returns nothing.	—
`ft_memcpy`	Copies n bytes from src to dst. No overlap. Returns dst.	Overlap = UB. NULL, NULL case.
`ft_memmove`	Like memcpy but safe with overlap.	`The copy direction depends on dst < src` (see 7.2).
`ft_strlcpy`	**Copies up to size-1 chars and always** puts '\0' (if size > 0).	*Returns strlen(src) (what it attempted to* copy), even if it is truncated. size == 0: copies nothing.
`ft_strlcat`	`Appends src to the end of dst without exceeding size` bytes in total.	Returning `min(size, strlen(dst)) +` strlen(src). See 7.2.
`ft_toupper / tolower`	Converts letters only; everything else remains the same.	Parameter and return type int.
`ft_strchr`	First occurrence of (char)c.	It can search for '\0': returns a pointer to the terminator.
`ft_strrchr`	Last occurrence.	Same: '\0' counts.
`ft_strncmp`	Compares at most n chars; stops at the first '\0'.	unsigned char. With n == 0, returns 0.
`ft_memchr`	Searches for a byte in n bytes; does not stop at '\0'.	Compare as unsigned char. Returns void *.

Function	What it must do	Traps and details they ask about
`ft_memcmp`	Compares n bytes; it does not stop at '\0'.	Return the difference between the first bytes that differ, as unsigned char.
`ft_strnstr`	Searches for needle in haystack, examining at most len chars.	Empty needle → return haystack. Never read beyond len or the '\0'.
`ft_atoi`	Skips whitespace (' ' and \t \n \v \f \r), optional sign, digits up to the first non-digit.	"--5" → 0. Overflow = UB. See 7.2.
`ft_calloc`	Reserve nmemb * size bytes initialized to zero.	Multiplication overflow → NULL. malloc failure → NULL.
`ft_strdup`	Copy the string into newly allocated memory.	Allocate len + 1. NULL if malloc fails.

7.1b Part 2 · additional functions

Function	What it should do	Traps and details they ask about
`ft_substr`	New string with up to len chars starting from start.	start >= strlen(s) → return string `reserved empty (not NULL, do not read out of bounds` greater than the remainder → truncate.
`ft_strjoin`	Concatenate s1 and s2 into newly allocated memory.	Allocate len1 + len2 + 1. Do not modify s1/s2.
`ft_strtrim`	Remove from the beginning and the end the chars that are in set.	Does not touch those in the middle. Everything trimmed → reserved empty string. Empty set → copy.
`ft_split`	Split by a delimiter into an array of strings terminated in NULL.	Consecutive delimiters / at the beginning / at the end, **empty string, free everything if a** malloc fails. See 7.2.
`ft_itoa`	Integer → string.	0, negatives, INT_MIN (you cannot do -n with an int). See 7.2.
`ft_strmapi`	New string applying f(i, s[i]) to each char.	`Returns a new string; f receives unsigned` int and char.
`ft_striteri`	Applies f(i, &s[i]) in place.	Does not allocate memory or return anything. f receives a char *.
`ft_putchar_fd / putstr_fd / putendl_fd`	They write to the file descriptor fd with write.	Decide what you do with NULL in putstr and be coherent. putendl adds '\n'.
`ft_putnbr_fd`	Writes an integer to fd.	INT_MIN: do not perform -n on an int.

7.1c Bonus · linked lists

```
typedef struct s_list
{
void *content; /* what the node stores (you do not copy it) */
struct s_list *next; /* next node, NULL at the end */
} t_list;
```

Function	What it does	Key detail
`ft_lstnew`	Creates a node with that content and next = NULL.	Stores the content pointer; it does not duplicate it. NULL if malloc fails.
`ft_lstadd_front(t_list **lst, t_list *new)`	Puts new at the beginning.	new->next = *lst; *lst = new; · receives **t_list ** because it must change the** caller's head pointer.
`ft_lstsize`	Count nodes.	Empty list (NULL) → 0.
`ft_lstlast`	Return the last node.	Empty list → NULL.

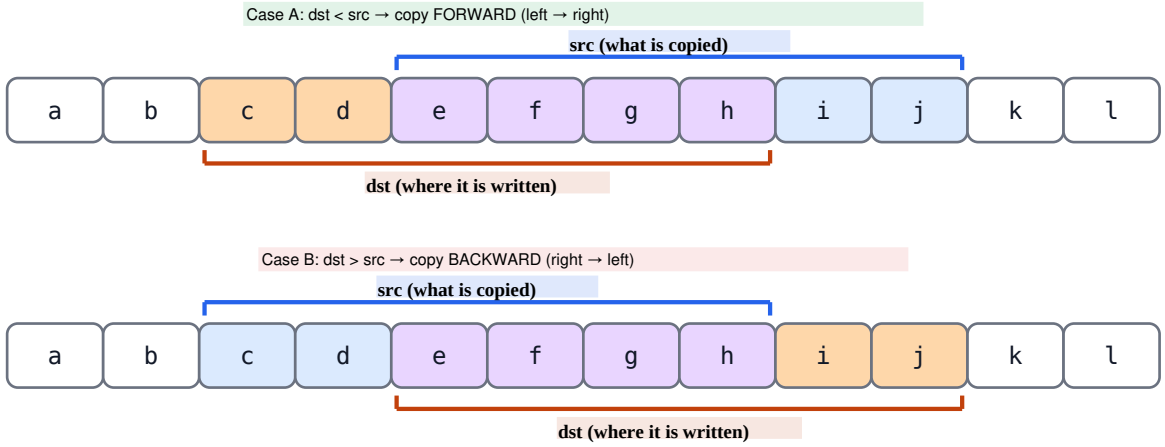
Function	What it does	Key detail
`ft_lstadd_back`	Adds to the end.	If *lst is NULL, new becomes the head.
`ft_lstdelone(lst, del)`	Calls del(lst->content) and does free(lst) .	It does not follow next or free the content on its own account.
`ft_lstclear(lst, del)`	`Deletes all nodes and leaves *lst` = NULL .	Save next before freeing.
`ft_lstiter(lst, f)`	Applies f to the content of each node.	Does not modify the structure.
`ft_lstmap(lst, f, del)`	Creates a new list with f(content) of each node.	Does not touch the original. If it fails: frees what was created (see 7.2).

7.2 In-depth analysis of the difficult ones

I compiled all this code with -Wall -Wextra -Werror and compared it against libc (with AddressSanitizer + **UBSan, with no errors or leaks). It is written to be understood; adapt it to the Norm (25 lines,** etc.) and, above all, write it in your own words. What you should take away is the reasoning, not the text.

ft_memmove: the overlapping-memory problem

If the source and destination overlap, always copying «from left to right» may overwrite source bytes before they have been read. The solution is to choose the direction based on which one comes first in memory.



Case A: copy forward, and each source byte is read before the destination overwrites it. Case B: forward you would overwrite the last bytes of src before reading them; you must copy from the end.

```
void *ft_memmove(void *dst, const void *src, size_t n)
{
unsigned char *d;
const unsigned char *s;

if (!dst && !src)
return (dst);
d = (unsigned char *)dst;
s = (const unsigned char *)src;
if (d < s)
while (n--) /* case A: forward */
            *d++ = *s++;
else
while (n--) /* case B: backward */
d[n] = s[n];
return (dst);
}
```

Trick

With d == s, the branch does not matter. And memmove is not “slower on purpose”: it only has to decide the direction.

If the evaluator asks “why not always use memcpy?”, the answer is: memcpy guarantees nothing with overlap (UB).

ft_strlcpy: the return value

```
size_t ft_strlcpy(char *dst, const char *src, size_t size)
{
size_t i;
size_t len;

len = ft_strlen(src);
if (size == 0)
return (len);
i = 0;
while (src[i] && i < size - 1)
    {
dst[i] = src[i];
        i++;
    }
dst[i] = '\0';
return (len);
}
```

- Copies at most size - 1 characters and always leaves a '\0' (if size > 0). That is why it is safer than strncpy, which can leave the string unterminated.
- Returns the length of src , not the amount copied. This allows the caller to detect truncation: if the result ≥ size, it was truncated.
- With size == 0, dst is not touched (not even the '\0'); only strlen(src) is returned .

ft_strlcat: the trickiest one

```
size_t ft_strlcat(char *dst, const char *src, size_t size)
{
size_t dlen;
size_t slen;
size_t i;

slen = ft_strlen(src);
dlen = 0;
while (dlen < size && dst[dlen]) /* strlen of dst, but WITHOUT going past size */
dlen++;
if (dlen == size) /* there is no '\0' within size */
return (size + slen);
i = 0;
while (src[i] && dlen + i < size - 1)
    {
dst[dlen + i] = src[i];
        i++;
    }
dst[dlen + i] = '\0';
return (dlen + slen);
}
```

- Return value = the length the result would have had if there had been room: min(size, strlen(dst)) + strlen(src). Just like strlcpy: it allows truncation to be detected.
- The length of dst is measured with the size limit, not with strlen: if dst does not have '\0' within the size bytes, strlen would read outside the area (UB).
- If size <= strlen(dst), nothing is written and size + strlen(src) is returned.

ft_atoi

```
int ft_atoi(const char *s)
{
long r;
```

```
int sign;

r = 0;
sign = 1;
while (*s == ' ' || (*s >= 9 && *s <= 13)) /* whitespace: ' ' and \t\n\v\f\r */
        s++;
if (*s == '-' || *s == '+')
if (*s++ == '-')
sign = -1;
while (*s >= '0' && *s <= '9')
r = r * 10 + (*s++ - '0');
return ((int)(r * sign));
}
```

- Order: spaces → a sign → digits → stop at the first non-digit. "--5" gives 0 because after the first there is no digit sign.
- Overflow: with very large numbers, the signed long overflows and this is UB (as in the original atoi too).

Accumulating in long only postpones the problem. If they ask you, the honest answer is this: it is not defined; I can bound it, but libc does not defined; I can bound it, but libc does not define it either.

ft_calloc: the overflow you don't see

```
void *ft_calloc(size_t nmemb, size_t size)
{
unsigned char *p;
size_t i;

if (size != 0 && nmemb > SIZE_MAX / size) /* Would nmemb * size overflow? */
return (NULL);
p = malloc(nmemb * size);
if (!p)
return (NULL);
i = 0;
while (i < nmemb * size)
p[i++] = 0;
return (p);
}
```

Unsigned integer overflow is defined (it wraps around), but then nmemb * size would be a small number: you would allocate less than requested, and the caller would write out of bounds. The check nmem > SIZE_MAX / size is done with a division (which never overflows) to detect it beforehand.

ft_split: three parts and a cleanup rule

It breaks down into counting words → allocating the array → allocating and copying each word, and this is the place where the most memory is lost if a malloc fails halfway through.

```
static size_t count_words(const char *s, char c)
{
size_t n;

n = 0;
while (*s)
    {
while (*s == c)
            s++;
if (*s)
            n++;
while (*s && *s != c)
            s++;
    }
return (n);
}

static void *free_all(char **arr, size_t n) /* free what has already been allocated */
{
while (n--)
free(arr[n]);
free(arr);
return (NULL);
}
```

```
char **ft_split(const char *s, char c)
{
char **arr;
size_t i;
size_t len;

if (!s)
return (NULL);
arr = malloc((count_words(s, c) + 1) * sizeof(char *));
if (!arr)
return (NULL);
i = 0;
while (*s)
    {
while (*s == c)
            s++;
if (!*s)
break ;
len = 0;
while (s[len] && s[len] != c)
len++;
arr[i] = malloc(len + 1);
if (!arr[i])
return (free_all(arr, i)); /* failed: frees arr[0..i-1] and arr */
ft_strlcpy(arr[i++], s, len + 1);
s += len;
    }
arr[i] = NULL; /* array terminator */
return (arr);
}
```

- Cases I tested: " hola que tal " → 3 words; "" and " " → array with only the final NULL; a string without a delimiter → 1 word.
- How many mallocs: 1 (the array) + 1 per word. How many frees: one for each, and the array one at the end. The user frees them with a loop and then free(arr).
- The helpers are static: they do not clutter the global symbols of libft.a (see 4.3).

> **They ask you**
> • «If malloc for the 3rd word fails, what happens to the first 2?» → free_all frees them and I return NULL; no *leak*.
> • «Why do you allocate count + 1?» → the last pointer is NULL, so the user knows where the array ends.

ft_itoa: INT_MIN and length calculation

```
static size_t num_len(long n)
{
size_t len;

len = (n <= 0); /* 0 and negative numbers need 1 extra char ('0' or '-') */
while (n)
    {
n /= 10;
len++;
    }
return (len);
}

char *ft_itoa(int n)
{
long nb;
size_t len;
char *s;

nb = n; /* long BEFORE negating: -INT_MIN does not fit in int */
len = num_len(nb);
s = malloc(len + 1);
if (!s)
return (NULL);
s[len] = '\0';
```

```
if (nb == 0)
s[0] = '0';
if (nb < 0)
    {
s[0] = '-';
nb = -nb;
    }
while (nb)
    {
s[--len] = '0' + nb % 10; /* filled from right to left */
nb /= 10;
    }
return (s);
}
```

I tested 0, 1, -1, 42, -42, 100, -100, INT_MAX, INT_MIN against snprintf: identical. The key: converting to long before doing -nb; with int, -INT_MIN is signed overflow (UB).

ft_lstdelone, ft_lstclear and ft_lstmap

```
void ft_lstdelone(t_list *lst, void (*del)(void *))
{
if (!lst)
return ;
if (del)
del(lst->content); /* the content is freed by whoever knows how: del */
free(lst); /* I free the node myself */
}

void ft_lstclear(t_list **lst, void (*del)(void *))
{
t_list *next;

if (!lst)
return ;
while (*lst)
    {
next = (*lst)->next; /* save BEFORE freeing: otherwise, use-after-free */
ft_lstdelone(*lst, del);
*lst = next;
} /* upon exit, *lst == NULL: the head remains NULL */
}
```

```
t_list *ft_lstmap(t_list *lst, void *(*f)(void *), void (*del)(void *))
{
t_list *head;
t_list *tail;
t_list *node;
void *c;

head = NULL;
tail = NULL;
while (lst)
    {
c = f(lst->content); /* new content (the original is not modified) */
node = ft_lstnew(c);
if (!node)
        {
del(c); /* the newly created content does not have a node yet */
ft_lstclear(&head, del); /* and the nodes already created */
return (NULL);
        }
if (!head)
head = node;
else
tail->next = node;
tail = node; /* remembering the tail avoids traversing the list each time */
lst = lst->next;
    }
return (head);
}
```

The key idea

¿Por qué t_list ** ? En C los argumentos se pasan por valor . Si ft_lstclear recibiera t_list * , pondría a NULL su copia local y el llamador seguiría con un puntero colgante. Con t_list ** modifica la variable del llamador. Mismo razonamiento en ft_lstadd_front y ft_lstadd_back .

Te lo preguntan

• ¿Por qué lstdelone recibe del en vez de hacer free del contenido? → No sabe qué es el contenido (un string, un struct con más punteros...).

• ¿Qué pasa en lstmap si falla el 4º nodo? → del del contenido nuevo + lstclear de los 3 nodos ya creados; *devuelvo NULL; la lista original queda intacta.*

8 · Banco de preguntas de defensa

Léelas en voz alta y contéstalas sin mirar . Si tu respuesta tiene el «porque», va bien; si solo it has the “what”; go back to the indicated chapter. The answers are phrased to be spoken in 15–20 seconds.

8.1 Library and compilation

Question	Answer that demonstrates that you understand it
What is libft.a?	A static library: an archive created with ar that groups my .o files and an index of symbols. It has no main; it is linked with another program. (1.1)
Why ar and not cc to create it?	Because it is neither an executable nor a single .o: it is a file that contains several .o files. ar is the tool that creates archives.
What does the “s” in ar rcs mean?	It writes/updates the symbol index (the same as ranlib), which the linker uses to find functions quickly.
Difference between compiling and link?	Compiling translates each .c into an .o while checking prototypes; linking combines the .o files and resolves each symbol with its definition. (1.2, 3.2)
What error does a function without a prototype produce? And without a definition?	Without a prototype: implicit declaration (compilation). With a prototype but without a definition: undefined reference (linking).
Why one .c file per function?	When linking with an .a, only the necessary members are extracted; with a .c per function, the executable only contains what it uses. (1.4)
How do I use libft in another project?	cc main.c -I. -L. -lft: -I for the header, -L for the library folder, -lft for libft.a, and after the files that use it.
What does -c do?	Compiles and assembles but does not link: generates an .o. Without it, cc tries to create an executable and fails due to the lack of main.

8.2 Memory

Question	Answer that demonstrates that you understand it
What is a segfault?	Access to unmapped memory or without permission; the MMU detects it, the kernel sends SIGSEGV and the process dies (exit 139). (5.4)
What is a double free?	Calling free twice with the same pointer. It corrupts the heap; glibc usually aborts. Avoid it with p = NULL after freeing. (5.5)
What is a memory leak?	Allocating memory and losing all pointers to it without freeing it. Detected with valgrind or AddressSanitizer. (5.6)
What is the difference between stack and heap?	Stack: automatic, for local variables, limited. Heap: manual with malloc/free, large, lasts until you free it. (5.1)
What does malloc return if it fails and what do you do?	NULL. You need to check it and, if other things had already been allocated, free them before returning NULL.
What does free(NULL) do?	Nothing: it is valid according to the standard.
What is a dangling pointer?	A pointer that points to memory that has already been freed or is out of scope. Using it is use-after-free.
What is a buffer overflow?	Writing beyond the end of a block. On the heap, this is usually silent (tested: exit 0) and corrupts neighboring data.
Why does s[0]='H' fail in char *s="hello"?	Because the literal resides in .rodata (read-only). With char s[]="hola" there is a modifiable copy on the stack.
How do you check that it doesn't do you have leaks?	A main with normal and failure cases for all functions, compiled with -fsanitize=address (or valgrind) and checking that there are no leaks or errors.

8.3 C, UB and function details

Question	Answer that demonstrates that you understand it
What is undefined behavior?	Code for which the standard does not define a result: it may work, fail, or break when changing compilers. The compiler assumes that this does not occur. (6.2)
Why is overflowing an int UB but not a size_t?	The standard defines unsigned overflow (modulo 2 n) and leaves it undefined for signed types.
Why is ft_itoa(INT_MIN) tricky?	`-INT_MIN does not fit in an int (signed overflow = UB). It is converted to long` before negation.
Why size_t?	It is the unsigned type for sizes; it represents any object size. Beware of i >= 0 with unsigned types: it is always true.
Why compare as unsigned char?	The signedness of char is implementation-dependent; if signed, 0xFF would be negative and the ordering would be wrong (memcmp 254 versus –2).
What is void *?	A generic pointer: it can point to anything, but it cannot be `dereferenced or used for arithmetic (standard). It is converted to unsigned cha` to work byte by byte.
memcpy or memmove?	memcpy does not allow overlap (UB); memmove does, choosing the copy direction according to dst < src . (7.2)
strlcpy vs strncpy?	strlcpy always terminates with '\0' and returns the length of src (allowing detection of truncation); strncpy may leave the string unterminated and pads it with zeros.
Why doesn't strlcat use strlen(dst)?	Because dst may not contain '\0' within size ; it must be measured with a limit to avoid read out.
What is static used for in a function?	Limits its visibility to its file: it is not a global symbol of libft.a, so it does not clash with user names.
Why does lstclear receive t_list **?	Because C passes by value: to set the head to NULL in the caller, it needs its address. (7.2)
What are del and f in lists?	Function pointers: they allow the list to be generic; I don't know what's in content, so I delegate to the caller how to free or transform it.
What happens with ft_strlen(NULL)?	Segfault, just like the original: it is UB. Imitating it is defensible; what matters is knowing how to explain why.
What does ft_lstnew store in content?	The pointer passed to it; it does not make a copy. Whoever created it remains responsible from its memory.

9 · Norm and subject rules (summary for review)

> **Note**
> This is a summary of the Norm and the subject as they usually are. Versions change: the source of truth is your current norminette and subject. If anything here does not match them, they take precedence.

9.1 Norm: what is violated most often

- Functions of 25 lines maximum, 80 columns per line, 5 functions per file, 4 parameters at most, 5 variables per function.
- Variables declared at the beginning of the function and one per line; without declaring and initializing at the same time in the majority of cases.
- Prohibited: for, do...while, switch, goto, ternary operator; non-constant global variables.
- Indentation with TAB; no comments inside functions; a line break after each declaration of function.
- Headers with guards. Every norminette error = project suspended: always run it before deliver .

9.2 libft Subject: what is checked

- Only certain external functions are allowed (generally malloc, free, and write): check the list of your subject .
- Makefile with NAME, all, clean, fclean, re (and bonus), compiling with -Wall -Wextra -Werror, without relinking.
- The library is generated with ar. The subject prohibits libtool (check this in your version).
- Bonus: it is evaluated only if the mandatory part is perfect.
- The repository must contain what is requested: neither binaries (.o, libft.a) nor unrelated test files.

10 · Defense protocol

10.1 Checklist before entering (with what you should see)

Command	Expected result
`make fclean && make`	Compile everything, without any warnings.
`make`	make: Nothing to be done for 'all'. → no relinking.
`touch ft_strlen.c && make`	Recompile only that file and rebuild libft.a.
`touch libft.h && make`	Recompile all the .o files.
make bonus (twice)	The second one: Nothing to be done.
`make clean, make fclean, make re`	They delete what they need and rebuild from scratch (also test run clean twice in a row).
`nm -g libft.a | grep ' T '`	Only the public functions (the static helpers should not appear).
`norminette`	Everything OK.
Your main with edge cases + `-fsanitize=address,undefined`	Zero errors and zero leaks.
`git status`	Clean: no .o, no libft.a, no test files accidentally uploaded.

10.2 During the defense

- Think out loud. “Make compares dates, so if I touch the header and the .o doesn’t list it...” demonstrates reasoning even when you are missing a piece of information.
- Read the errors aloud and classify them by stage (preprocessing / compilation / linking / execution).
- If they ask you to modify something: state your plan in one sentence, keep it small, compile, test. Don’t rewrite everything.
- If they tell you “what if...?”, go back to the underlying rule of the chapter (dates, symbols, owner of the malloc...).
- Have a test main.c ready outside the repository, and the terminal already open in the folder libft.
- Don’t become defensive if they find a bug: acknowledge it, explain it, and fix it. That is valued.

10.3 Self-assessment: what if...? (answer aloud)

Question	Where to review it
What if I rename libft.a to libft.c in NAME?	2.9
What if I move clean before all? What if I remove .PHONY?	2.3, 2.7
What if I change re to “all fclean”? What if I run make -j?	2.3, 2.8
What if I remove the header from the .o rule? What if I add -g to CFLAGS?	2.6, 2.8
What if you use wildcard? What if I remove -c?	2.8
What if I add a new function? What if I request a debug mode?	2.10
What if I call a function without a prototype? What if the definition is missing?	3.2
What if the header has no guards? What if I put a function with a body in the .h?	4.2
What if I free something twice? What if I don't free it?	5.5, 5.6
What if I write to a string literal?	5.3, 5.4

Question	Where to review it
What if ft_itoa receives INT_MIN? And ft_calloc(SIZE_MAX, 2)?	6.4, 7.2
What if malloc fails halfway through ft_split? And ft_lstmap?	7.2
What if the source and destination overlap in memcpy / memmove?	7.2

10.4 Final 30-minute lab

If you only have half an hour, do this: each step reinforces an idea that you will then be able to explain without thinking.

1. NAME = libft.c → make, file libft.c, cc main.c libft.c, make fclean. Look at each result. (2.9)
2. Move clean above all and run make. Put it back in its place. (2.3)
3. Remove .PHONY, create a clean file, and run make clean. (2.7)
4. Remove libft.h from the %.o rule, run touch libft.h and make. Put it back. (2.6)
5. Change to re: all fclean and see what remains. Try make -j8 re with the original. (2.8)
6. Measure the relink: run make twice; make bonus twice. (2.5)
7. Cause a segfault, double free, leak, use-after-free, and overflow in 5 three-line files; run them with and without -fsanitize=address . (5.3)
8. Header without guards included twice, and a function with a body in a .h included from two .c files. (4.2)
9. UB: x + 1 > x with INT_MAX and memcmp with 0xFF vs 0x01. (6.2, 7.0)
10. Run your complete libft through a main with edge cases and -fsanitize=address,undefined.

Appendix A · Command cheat sheet

For...	Command
See what make would do without executing	`make -n`
Force rebuilding everything	`make -B`
See why it rebuilds each target	`make --debug=b`
List the library members	`ar t libft.a`
List symbols (T = defined, U = undefined)	nm libft.a · nm -g libft.a | grep ' T '
See what a file is	`file libft.a`
View the code after the preprocessor	`cc -E fichero.c`
Generate assembly	`cc -S fichero.c`
Compile a main with your library	`cc main.c -I. -L. -lft -o prueba`
Debug with sanitizers	`cc -g -O0 -fsanitize=address,undefined main.c libft.a -I. -o prueba`
Look for leaks with valgrind (Linux)	`valgrind --leak-check=full --show-leak-kinds=all ./prueba`
Search for leaks in macOS	`leaks --atExit -- ./prueba`
Locate a segfault	gdb ./prueba → run → bt
Check the standard	`norminette`
See the exit code of the last program	echo $? (139 = segfault, 134 = abort)

Appendix B · One-line glossary

Term	Meaning
Archive (.a)	File that groups several .o files with a symbol index.
Object (.o)	Machine code from a .c file with unresolved references.
Symbol	Name of a function or global variable visible to the linker.
Linker (ld)	Links objects and libraries, resolves symbols, and produces the executable.
Prototype	Declaration of a function (name, parameters, return type) without a body.
Compilation unit	A .c file with all headers already included: what cc compiles in one go.
Guard	The #ifndef/#define/#endif mechanism that prevents a header from being processed twice.
Target / prerequisite	In make: what is built / what it depends on.
Pattern rule	A rule with % that describes how to make a family of files.
Phony	A make target that is not a file.
Relink	Running compilation/archiving again without anything having changed.
Stack / heap	The function's automatic memory / dynamic memory managed with malloc and free.
Segfault	Access to invalid memory → SIGSEGV.
Leak	Reserved memory with no pointer allowing it to be freed.
Dangling pointer	Pointer to memory that has already been freed or is out of scope.
UB	Behavior for which the standard does not define a result.
Sanitizer	Compiler instrumentation that detects runtime errors (ASan, UBSan).

To finish

The best sign that you have mastered libft is not remembering every answer, but being able to deduce it. If, faced with any "¿y si...?" you are able to say which rule applies (make compares dates, the linker resolves symbols, every malloc has an owner, UB has no guarantees), you will be more than ready. Best of luck with the defense.